

SE-12-08: Communication Issues in Software Engineering

Effective communication is a core technical and ethical requirement in the **NSW Software Engineering 11–12 Syllabus (2022)**. Outcome **SE-12-08** emphasises that a software solution's success is determined not just by the code's efficiency, but by how well the developer manages relationships and information flow with stakeholders.

+2

1. Involving and Empowering the Client

Empowerment occurs when the client is treated as an active participant in the development process rather than a passive recipient.

- **Active Involvement:** Developers should meet regularly with the client to clarify requirements and demonstrate progress.
- **Transparency:** Clearly defining **System Boundaries**—what is included and, crucially, what is NOT included—empowers the client to make informed decisions about project scope.
- **User Interface (UI) Prototypes:** Using storyboards and wireframes early in the design phase allows clients to "see" the solution and provide direction before significant coding begins.

2. Enabling and Responding to Feedback

Feedback loops are critical to the iterative nature of modern software development, particularly in **Agile** and **WAgile** approaches.

- **Multi-Channel Feedback:** Developers should seek input from a variety of sources, including peers, teachers, and end-users.
- **Formal Reporting:** Feedback should be analysed and synthesised into a formal report to document how it influenced the final software solution.
- **Iterative Refinement:** High-quality projects demonstrate that the software was refined based on the testing and feedback received, rather than ignoring user critiques.

3. Negotiation of Requirements and Changes

Negotiation is the process of balancing the client's "wish list" against technical, financial, and time constraints.

- **Feasibility Constraints:** When a client requests a feature, the engineer must evaluate it against **Scheduling** (time), **Technical** (skills/software), and **Financial** (cost) feasibility.
- **Compromise and Alternatives:** If a requirement is unfeasible, the developer should suggest alternatives—such as an "Innovative Feature" or a "Prototype Design"—that meet the user's intent without compromising the project deadline.
- **Documentation of Change:** All negotiated changes should be reflected in the updated **Requirements Analysis** to maintain a clear "source of truth" for the project.

Summary Table: Communication Strategies

Communication Goal	Key Syllabus Tools/Methods	Rationale
Empowerment	Storyboards, System Boundaries +4	Ensures client expectations align with project reality.
Feedback	Feedback Analysis Reports, Peer Reviews +1	Identifies usability issues and logic errors early.
Negotiation	Feasibility Analysis, Gantt Charts +4	Justifies why certain features can or cannot be implemented.

6-Mark HSC Model Response

Question: *Explain how a software engineer can use communication strategies to effectively manage project changes and involve a client throughout the development process.*

Model Answer: Effective project management relies on involving and empowering the client to ensure the software meets defined needs. A software engineer initiates this through a **Feasibility Analysis**, clearly communicating time and technical constraints to the client early in the project. By defining **System Boundaries**, the engineer manages expectations by explicitly stating what the system will not include, which prevents "scope creep" and empowers the client to prioritise essential functional requirements.

Furthermore, enabling a structured **feedback loop** is vital for iterative refinement. By using **Storyboards and UI sketches**, the developer allows the client to provide feedback on the visual design before the back-end engineering is finalised. When feedback necessitates a change, the engineer must engage in **negotiation**, using a **Gantt chart** to demonstrate how the new requirement impacts the schedule. This evidence-based approach to communication ensures that quality is maintained through continual checking while fostering a transparent relationship between the developer and the stakeholder.